

Monorepo PoC

Fra spredte SNAPSHOTS til deterministiske bygg

Kontekst

Dette er siste dag min på Lovdata. Jeg gir dere et problem og et forslag.

Problemet: Backend-bygget er ikke-deterministisk.

Forslaget: Samle alt i ett monorepo med pinned versjoner.

Advarsel: Dette er et PoC. Det er dristig. Men det er riktig.

Nåværende struktur

```
lovdata-main      (repo 1)
lovdata-objects   (repo 2)
lovdata-services  (repo 3)  ] → lovdata-pro-v2 (repo 4)
```

Fire repoer. Fire ulike commit-historikker. **Ingen garantert sammenheng.**

- En endring i `objects` krever: commit → push → deploy til Nexus → rebuild
- En CI-pipeline er en serie *antakelser*, ikke *fakta*
- Umulig å svare på: "Hva kjørte vi i prod 3. januar?"

Problemet i én linje

```
<version>1.0.0-SNAPSHOT</version>
```

Dette er ikke en versjon.

Dette er et løfte om at *noen* vil levere *noe* – på et *tidspunkt*.

Hva betyr egentlig **SNAPSHOT** ?

```
<parent>
  <groupId>no.lovdata</groupId>
  <artifactId>lovdata-main</artifactId>
  <version>1.0.0-SNAPSHOT</version>  <!-- ← hvilket bygg? -->
</parent>

<dependencies>
  <dependency>
    <groupId>no.lovdata</groupId>
    <artifactId>lovdata-objects</artifactId>
    <!-- versjon arves → 1.0.0-SNAPSHOT -->
  </dependency>
  <dependency>
    <groupId>no.lovdata</groupId>
    <artifactId>lovdata-services</artifactId>
    <!-- versjon arves → 1.0.0-SNAPSHOT -->
  </dependency>
</dependencies>
```

Maven henter **nyeste tilgjengelige** SNAPSHOT fra Nexus – ikke den du brukte sist.

Den reelle konsekvensen

```
commit abc123 ← "Fiks søkefeil"
```

Bygd mandag: lovdata-services 1.0.0-SNAPSHOT @ hash a1b2c3

Bygd tirsdag: lovdata-services 1.0.0-SNAPSHOT @ hash d4e5f6

To forskjellige artifakter. Samme commit-SHA.

Du kan ikke reproducere et bygg.

Du kan ikke bisecte en feil.

Du kan ikke stole på CI-loggen.

Sporbarheten er ikke redusert – den er eliminert.

"Making impossible states impossible" – på build-nivå

I kode vet vi dette er feil:

```
// Feil: streng kan være hva som helst  
fun processDocument(version: String) { ... }  
  
// Riktig: typen utelukker ugyldige tilstander  
fun processDocument(version: SemanticVersion) { ... }
```

I byggsystemet gjør vi det motsatte:

```
<!-- "Hvilken versjon av lovdata-services?" -->  
<!-- "Ja." -->  
<version>1.0.0-SNAPSHOT</version>
```

En SNAPSHOT-versjon er `String` der vi trenger `SemanticVersion`.

Kontrasten: Elm vet hva den har

```
{
  "dependencies": {
    "direct": {
      "elm/core": "1.0.5",
      "elm/html": "1.0.0",
      "elm/json": "1.1.3"
    }
  }
}
```

Prøv å introdusere en SNAPSHOT i `elm.json`. Det går ikke. Kompilatoren nekter.

Elm-frontenden er deterministisk av design.

Backend-en vår er ikke-deterministisk av vane.

Samme commit. Samme maskin. Samme dag. Elm gir alltid samme resultat. Maven gjør ikke det.

Løsningen: Monorepo

"If you can't reproduce a build from a single commit,
you don't have a build — you have a wish."

Prinsippet: Én git-commit = én komplett sannhet.

```
lovdata/
├── pom.xml                ← parent POM, eier det hele
├── lovdata-objects/
│   ├── pom.xml
│   └── src/
├── lovdata-services/
│   ├── pom.xml
│   └── src/
└── lovdata-pro-v2/
    ├── pom.xml
    └── src/
```

Ingen SNAPSHOT-avhengigheter på tvers – alt er lokale modulreferanser.

Maven Multi-Module: Parent POM

```
<!-- pom.xml (rot) -->
<project>
  <groupId>no.lovdata</groupId>
  <artifactId>lovdata-parent</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>

  <modules>
    <module>lovdata-objects</module>
    <module>lovdata-services</module>
    <module>lovdata-pro-v2</module>
  </modules>
</project>
```

```
<!-- lovdata-pro-v2/pom.xml -->
<dependency>
  <groupId>no.lovdata</groupId>
  <artifactId>lovdata-objects</artifactId>
  <version>${project.version}</version> <!-- løses lokalt, ikke fra Nexus -->
</dependency>
```

Maven løser avhengighetene **lokalt i samme bygg**. Ingen Nexus-rundtur.

Alternativt: Gradle med Version Catalogs

`gradle/libs.versions.toml` – ett sted for alle versjoner:

```
[versions]
kotlin = "2.0.21"
ktor = "3.0.3"
coroutines = "1.9.0"

[libraries]
ktor-server-core = { module = "io.ktor:ktor-server-core", version.ref = "ktor" }
ktor-server-netty = { module = "io.ktor:ktor-server-netty", version.ref = "ktor" }
kotlinx-coroutines = { module = "org.jetbrains.kotlinx:kotlinx-coroutines-core", version.ref = "coroutines" }

[bundles]
ktor = ["ktor-server-core", "ktor-server-netty"]
```

```
// build.gradle.kts
dependencies {
    implementation(libs.bundles.ktor)
    implementation(libs.kotlinx.coroutines)
}
```

Én PR for å bumpe en versjon. Diff er tydelig. CI er deterministisk.

Maven Enforcer: Håndhev det maskinelt

Beholder Maven, men sperrer SNAPSHOTS i produksjonsbygg:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-enforcer-plugin</artifactId>
  <configuration>
    <rules>
      <requireReleaseDeps>
        <onlyWhenRelease>true</onlyWhenRelease>
        <message>SNAPSHOTS er ikke tillatt i produksjonsbygg</message>
      </requireReleaseDeps>
    </rules>
  </configuration>
</plugin>
```

Og ekstern BOM for JetBrains-familien:

```
<dependency>
  <groupId>io.ktor</groupId>
  <artifactId>ktor-bom</artifactId>
  <version>3.0.3</version> <!-- aldri SNAPSHOT -->
  <type>pom</type>
  <scope>import</scope>
</dependency>
```

Gevinster

	I dag	Monorepo
Reproduserbart bygg?	Nei	Ja
Atomiske commits?	Nei	Ja
Rollback én operasjon?	Nei	Ja
Bisectable CI-feil?	Vanskelig	Enkelt
"Hva kjørte vi i prod?"	Gjetning	<code>git log</code>
PR-diff lesbar?	Ikke alltid	Alltid

Atomiske commits: Én PR endrer `objects` og `pro-v2` samtidig – de er alltid i sync.

Rollback: `git revert <sha>` – én operasjon, hele systemet tilbake.

Hva er risikoen?

Migrering tar tid — men SNAPSHOT-problemet forsvinner ikke av seg selv.

Større repo — men verktøy som `mvn -pl lovdata-pro-v2` bygger bare det du trenger.

Mer koordinering i PR-er — men langt mindre enn å koordinere på tvers av 4 repoer.

"The goal of software architecture is to minimize the human resources required to build and maintain the required system."
— Robert C. Martin, Clean Architecture

Å jage SNAPSHOTs på tvers av 4 repoer er akkurat det motsatte.

PoC-forslaget

1. **Flytt** `lovdata-main`, `lovdata-objects`, `lovdata-services` og `lovdata-pro-v2` til én repo
2. **Gjør om** til Maven multi-module med lokal modul-resolving
3. **Legg til** Maven Enforcer som blokkerer SNAPSHOTs i CI
4. **Verifiser** med én `mvn clean install` at alt er grønt

Dette er ikke en stor omskrivning. Det er en restrukturering.

Koden er den samme. Byggepipelinen blir ærlig.

Takk

Tre og et halvt år på Lovdata.

Elm i frontend: deterministisk fra dag én.

Maven i backend: SNAPSHOT helt til slutt.

Dere vet hva som er riktig. 😊

Spørsmål?